

Prosta współbieżność w C++

Bartosz Palacz, Maciej Sidorski

Ograniczenia klasycznego `std::thread`



Architektura `std::async` i `std::future`

Producent (Dostawca):

Wątek pracujący w tle.
Wykonuje zadanie i
zapisuje jego wynik za
pomocą `std::async`.

Stan Współdzielony:

"Ukryta skrytka" na dane.
Automatycznie dba o
synchronizację, więc nie
musisz martwić się o
zablokowanie programu.

Konsument (Odbiorca):

Główny wątek używający
obiektu `std::future`.
Pozwala bezpiecznie
"wyciągnąć" wynik, gdy
ten będzie już gotowy.

std::future

```
#include <iostream>
#include <future>

int main() {
    std::future<int> bilet_po_wynik = std::async([] {return 42;});

    try {
        int wynik = bilet_po_wynik.get();
        std::cout << "Sukces! Wynik: " << wynik << '\n';
    }
    catch (const std::exception& e) {
        std::cerr << "Błąd w tle: " << e.what() << '\n';
    }
}
```

- ❑ **Bilet po wynik:** Obiekt `std::future` to nasza gwarancja, że otrzymamy wynik z zadania działającego w tle.
 - **.wait():** Czeka na zakończenie obliczeń, ale jeszcze nie wyciąga z nich wyniku.
 - **.get():** Pobiera gotową wartość. **Uwaga:** Blokuje program do czasu, aż wynik będzie gotowy i można go użyć tylko raz
 - **Bezpieczne błędy:** Jeśli wątek w tle rzuci wyjątek, to złapiemy go właśnie podczas wywoływania `.get()`.
-

```
#include <iostream>
#include <vector>
#include <numeric>
#include <future>

int policz_sume(const std::vector<int>& dane) {
    return std::accumulate(dane.begin(), dane.end(), 0);
}

int main() {
    std::vector<int> duze_dane(1'000'000, 1);

    std::future<int> bilet = std::async(std::launch::async, policz_sume, std::ref(duze_dane));
    std::cout << "Suma to: " << bilet.get() << '\n';

    return 0;
}
```

std::async

- **Pełna automatyzacja:** Kompilator sam zarządza cyklem życia wątku nie musisz pamiętać o .join() jak w przypadku std::thread.
 - **Wszystko w jednym:** Automatycznie pakuje parametry i wynik do stanu współdzielonego.
 - **Łatwe przekazywanie danych:** Wygodnie przyjmuje argumenty, pozwala też na referencje (wymaga użycia std::ref).
 - ☐ **Zero rygorów:** Koniec z ręcznym stawianiem mutexów przy prostym zwracaniu wyników!
-

Polityki uruchomieniowe

Kiedy i jak wykona się zadanie?

- **Wymuszenie `std::launch::async`** Zadanie startuje natychmiast w nowym, osobnym wątku w tle. Pełna równoległość.
- **Leniwie `std::launch::deferred`** Zadanie nie startuje od razu. Czeka na wywołanie `.get()` i dopiero wtedy wykonuje się w Twoim obecnym wątku (nie tworzy nowego).
- **Złota zasada:** Zawsze podawaj flagę `std::launch::async`, jeśli chcesz mieć pewność, że zadanie pobiegnie w tle. Bez tego system operacyjny sam zadecyduje i często wybierze opcję leniwą.

```
#include <iostream>
#include <future>
#include <thread>
#include <string>

void sprawdz_watek(const std::string& nazwa) {
    std::cout << nazwa << " - ID watku: " << std::this_thread::get_id() << '\n';
}

int main() {
    sprawdz_watek("Watek GLOWNY");

    auto bilet_async = std::async(std::launch::async, sprawdz_watek, "Zadanie ASYNC");

    auto bilet_defer = std::async(std::launch::deferred, sprawdz_watek, "Zadanie DEFERRED");

    std::cout << "Główny wątek prosi o wynik \n";

    bilet_defer.get();
    bilet_async.get();

    return 0;
}
```

Czym jest std::promise?

std::promise to ręczny kanał zapisu — tworzysz po stronie producenta, przekazujesz future konsumentowi, wartość ustawiasz kiedy chcesz. Dwa warianty: dla wyniku i dla błędu.

set_value_at_thread_exit() działa jak **set_value()**, ale odblokowuje future dopiero po zakończeniu wątku.
promise służy do czystej sygnalizacji — **set_value()** bez argumentu.

PRZEKAZANIE WARTOŚCI: SET_VALUE()

```
void calc(std::promise<int> p) {
    p.set_value(42);
}

// Główny wątek:
std::promise<int> p;
auto f = p.get_future();

std::thread t(calc, std::move(p));

int res = f.get(); // blokuje do set_value
```

PRZEKAZANIE BŁĘDU: SET_EXCEPTION()

```
void fail_task(std::promise<int> p) {
    try {
        throw std::runtime_error("Błąd");
    } catch (...) {
        p.set_exception(std::current_exception());
    }
}

// Główny wątek:
std::promise<int> p;
auto f = p.get_future();

std::thread t(fail_task, std::move(p));

try { f.get(); } // rethrow
catch (const std::exception& e) { }
```

| promise — opóźnienie gotowości

set_value_at_thread_exit()

Zapisuje wynik, ale odblokowuje future dopiero po całkowitym zakończeniu wątku i usunięciu zmiennych `thread_local`. Gwarantuje to pełne zwolnienie zasobów przed odczytem

```
void delayed_worker(std::promise<int> p) {  
    thread_local int temp = 0;  
    p.set_value_at_thread_exit(100);  
}  
  
// Główny wątek:  
std::promise<int> p;  
auto f = p.get_future();  
  
std::thread t(delayed_worker, std::move(p));  
t.join();
```


Częste błędy (std::future_error)

future_already_retrieved

Wielokrotne wywołanie get_future() na tej samej obietnicy.

```
std::promise<int> p;  
auto f1 = p.get_future();  
auto f2 = p.get_future(); // Błąd
```

promise_already_satisfied

Ponowne ustawienie wyniku w gotowym stanie.

```
std::promise<int> p;  
p.set_value(10);  
p.set_value(20); // Błąd
```

broken_promise

Promise zniszczone bez ustawienia wartości. Najczęściej: promise wychodzi z zakresu zanim zostanie wypełnione.

```
std::future<int> get_data() {  
    std::promise<int> p;  
    auto f = p.get_future();  
  
    // Błąd: zapomniano wywołać p.set_value()  
    return f;  
} // 'p' ginie przy wyjściu z funkcji → broken_promise  
  
auto f = get_data();  
f.get(); // rzuca błąd
```

Czym jest std::packaged_task?

std::packaged_task owija dowolny callable i automatycznie ustawia future gdy go wywołasz — zamiast ręcznego set_value() z promise, wystarczy zwykły return. Wyjątki też są wychwytywane automatycznie, bez try/catch.

AUTOMATYCZNY ZWROT: RETURN

```
std::packaged_task<int()> task([] {  
    return 100;  
});  
auto f = task.get_future();  
  
std::thread t(std::move(task));  
int res = f.get();
```

AUTOMATYCZNY WYJĄTEK: THROW

```
std::packaged_task<int()> task([]() -> int {  
    throw std::runtime_error("Awaria");  
});  
auto f = task.get_future();  
  
std::thread t(std::move(task));  
try { f.get(); }  
catch (const std::exception& e) { }
```

| packaged_task — valid() i reset()

valid() sprawdza aktywność stanu

współdzielonego (zwraca false po std::move).

Stan jest jednorazowy — ponowne wywołanie tej samej logiki wymaga użycia reset(), który alokuje nowy stan dla kolejnego wyniku.

```
std::packaged_task<int()> task([]{ return 7; });

task.valid();           // true
auto t2 = std::move(task);
task.valid();           // false – przeniesiony
t2.valid();             // true

t2();
t2.get_future().get();  // 7

t2.reset();             // nowy shared state
t2();
t2.get_future().get();  // 7, znowu
```

Porównanie: promise vs packaged_task

Oba mechanizmy służą do jednorazowego przekazania wyniku do `std::future`. Wybór zależy od tego, czy potrzebujemy ręcznej kontroli przepływu danych, czy prostej automatyzacji dla funkcji.

STD::PROMISE — PEŁNA KONTROLA

- **Cel:** Ręczne sterowanie z dowolnego miejsca.
- Wymaga ręcznego wywołania `set_value()`.

```
void worker(std::promise<int> p) {  
    p.set_value(5 + 7);  
}  
  
std::promise<int> p;  
auto f = p.get_future();  
  
std::thread t(worker, std::move(p));  
  
int res = f.get(); // 12
```

STD::PACKAGED_TASK — AUTOMATYZACJA

- **Cel:** Zlecanie gotowych funkcji do wykonania.
- Przechwytuje zwykły `return` i wyjątki.

```
int worker_func() {  
    return 5 + 7;  
}  
  
std::packaged_task<int()> task(worker_func);  
auto f = task.get_future();  
  
std::thread t(std::move(task));  
  
int res = f.get(); // 12
```